

Guía Pedagógica: Día 13 - Animaciones de Transición (View Transitions API)

Hoy vamos a implementar una de las características más modernas y espectaculares del desarrollo web actual: la **View Transitions API**.

¿Alguna vez has notado cómo en las aplicaciones móviles nativas (como iOS o Android), cuando tocas una imagen, esta "vuela" y se expande suavemente hasta convertirse en la imagen principal de la siguiente pantalla? Eso es exactamente lo que lograremos hoy al navegar de la Home al Detalle de una película.

Contexto Pedagógico: ¿Qué es View Transitions?

Históricamente, animar el cambio de una página a otra en la web era un dolor de cabeza. Requería librerías complejas y mucho cálculo matemático.

La **View Transitions API** es una función nativa de los navegadores modernos que toma una "foto" de la página actual, otra de la página nueva, y el navegador automáticamente crea una transición fluida entre ambas. Angular 17+ integró esta API directamente en su Router, ¡haciendo que activarla sea ridículamente fácil!

Paso 1: Activar View Transitions en el Router

Primero, debemos decirle al enrutador de Angular que queremos usar esta funcionalidad globalmente.

Abre el archivo **src/app/app.config.ts** y añade `withViewTransitions()` a la configuración del Router:

```
import { ApplicationConfig } from '@angular/core';
import { provideRouter, withViewTransitions } from
'@angular/router'; // Añadido
import { routes } from './app.routes';
import { provideHttpClient, withFetch } from
'@angular/common/http';
import { provideClientHydration } from '@angular/platform-
browser';

export const appConfig: ApplicationConfig = {
  providers: [
    // Activamos las transiciones de vista aquí 🙌
    provideRouter(routes, withViewTransitions()),
    provideClientHydration(),
    provideHttpClient(withFetch())
  ]
};
```

¡Con solo esa línea de código, TODA tu aplicación ahora tiene un efecto de fundido cruzado (fade) muy elegante al cambiar de página!

Paso 2: La Magia del "Morphing" (Conectando Elementos)

El fundido cruzado está bien, pero queremos el efecto "WOW". Queremos que el póster pequeño de la película en la Home "vuele" y se transforme en el póster grande de la vista de detalles.

Para que el navegador sepa qué elemento de la Página A se convierte en qué elemento de la Página B, debemos darles a ambos **exactamente el mismo nombre de transición** usando CSS: `view-transition-name`.

Como el nombre debe ser único por película, lo generaremos dinámicamente usando el ID de la película (ej. `movie-poster-12345`).

Modificar la Tarjeta (Página A - Origen)

Abre `src/app/shared/components/movie-card/movie-card.html` y añade el estilo en línea a la imagen del póster:

```
<!-- Añadimos [style.view-transition-name] a la imagen -->
<img
  [src]="posterUrl"
  [alt]="movie.title"
  class="movie-poster"
  [style.view-transition-name]="`movie-poster-` + movie.id">
```

Modificar el Detalle (Página B - Destino)

Abre `src/app/features/movie-details/movie-details.html` y añade exactamente el mismo estilo a la imagen del póster principal:

```
<!-- Añadimos [style.view-transition-name] a la imagen -->
<img
  [src]="`https://image.tmdb.org/t/p/w500` +
data.details.poster_path"
  [alt]="data.details.title"
  class="poster-img"
  [style.view-transition-name]="`movie-poster-` +
data.details.id">
```

Paso 3: Ajustes de Animación (Opcional pero Recomendado)

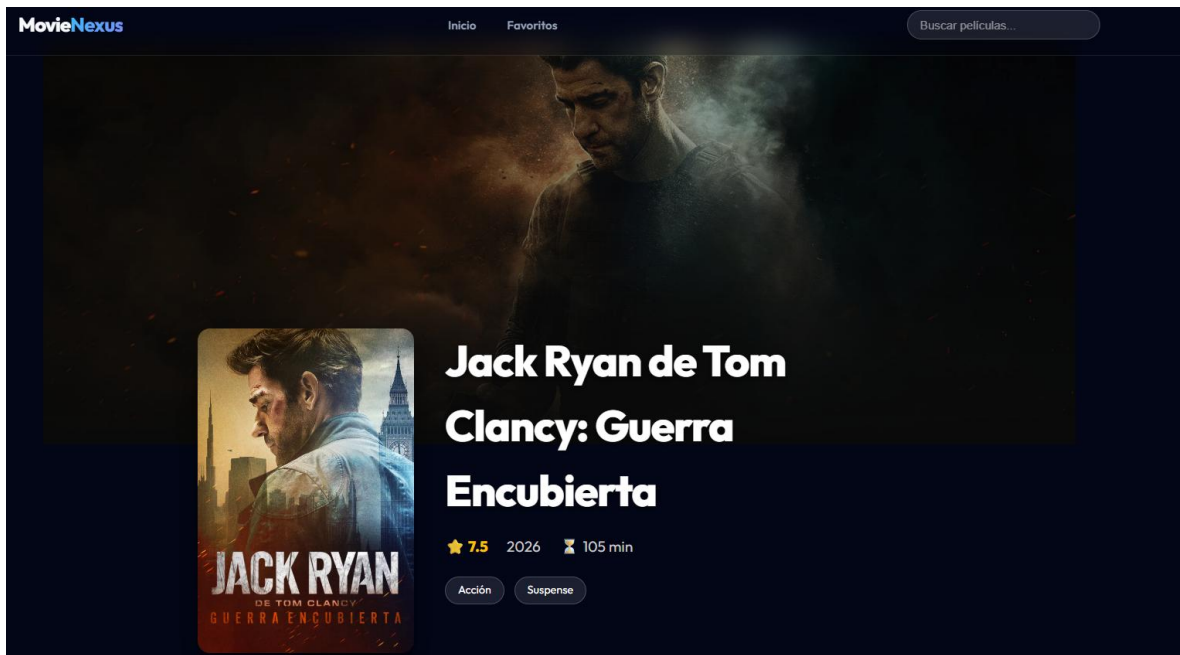
Angular y el navegador hacen el trabajo pesado, pero a veces queremos que la animación sea un poco más rápida o tenga un rebote (bounce). Podemos personalizar esto con CSS global.

Abre **src/styles.css** y añade esto al final:

```
/* =====  
VIEW TRANSITIONS API CUSTOMIZATION  
===== */  
  
/* Hacemos que todas las transiciones sean un poco más lentas y fluidas para  
apreciarlas */  
::view-transition-old(root),  
::view-transition-new(root) {  
  animation-duration: 0.4s;  
}  
  
::view-transition-group(*) {  
  animation-duration: 0.4s;  
  animation-timing-function: cubic-bezier(0.4, 0, 0.2, 1);  
}
```

Paso 4: ¡Pruébalo!

1. Ve a tu Home (<http://localhost:4200>).-----> ng serve -o
2. Haz clic en el póster de cualquier película.
3. Observa cómo el póster **se despega de la grilla y vuela por la pantalla** hasta acomodarse en su posición final en la vista de detalles.
4. Si haces clic en el botón de "Atrás" del navegador, el póster volverá volando a su lugar original en la grilla.



¡Felicidades! Acabas de implementar una animación de nivel nativo en una aplicación web.

Prueba de Aprendizaje (Checklist)

1. **¿Qué función habilita la View Transitions API en Angular Router?** (Respuesta: `withViewTransitions()` dentro de `provideRouter()` en `app.config.ts`).
2. **Si tengo dos imágenes en diferentes páginas y quiero que se animen conectándose entre sí, ¿qué propiedad CSS debo asegurar que compartan?** (Respuesta: Deben compartir el mismo `view-transition-name`. Además, este nombre debe ser único en la pantalla en ese momento, por eso usamos el ID de la película).
3. **¿Qué ocurre con los elementos que NO tienen un `view-transition-name` específico?** (Respuesta: El navegador les aplica una animación por defecto, que generalmente es un "cross-fade" (fundido cruzado) suave sobre el elemento root (toda la página)).

Control de Versiones

Esta es una mejora masiva para la percepción de calidad de tu aplicación. Guárdala:

```
git add .
git commit -m "feat: implementar View Transitions API para navegacion fluida
estilo app nativa"
git push origin main
```

